

Sistemas Operativos - 20205

FCEFQyN - Universidad Nacional de Río Cuarto

Nota: Todos los proyectos deben realizarse sobre la última versión de edos (step08-process), disponible en <https://gitlab.com/marceloarroyo/edos-steps>.

Proyecto 1: Ejecución de programas ELF.

Edos actualmente almacena los contenidos de los archivos de usuario en arreglos de bytes en el archivo `efsfiles.c` el cual representa un RAM (embedded) filesystem enlazado con el resto del código y datos del kernel. Actualmente el script `mkefs.sh`, que construye el archivo `efsfiles.c` extrae la sección de texto (código) y datos globales (`.data`) mediante `objdump` en los `.bin` correspondientes. Esto hace que *edos* pierda información sobre ejecutables ya que las cabeceras del ELF se omiten.

1. Modificar `mkefs.sh` para generar `efsfiles.c` que contenga directamente el contenido del archivo ELF.
2. Modificar `user/Makefile` y `user/user.ld` para que los procesos de usuario se enlacen a partir de la dirección `0x1000` y el entry point sea la función `start`.
3. Modificar las funciones `exec()` y auxiliares (en `task.c`) del kernel para que cargue y ejecute binarios ELF teniendo en cuenta las secciones a cargar (texto y datos marcados con `LOAD` en el program header) y el punto de entrada del programa (en el ELF header).
4. Asignar los permisos `RXU` al área de código en memoria y `RWU` a las áreas de datos. Notar que actualmente *edos* no distingue el código de los datos en los archivos ejecutables representados en `efsfiles.c`.

Nota: El archivo `elf.h` contiene declaraciones del formato de la cabecera y de los program headers. Formato ELF: https://es.wikipedia.org/wiki/Executable_and_Linkable_Format

Proyecto 2: Shared Memory

Implementar un espacio de memoria compartido entre dos o más procesos. El área reservada para los bloques de memoria compartida deberá estar ubicada

lógicamente debajo del área de datos globales (encima del stack). Para eso deberán implementar en edos un módulo `sharedmem.c` que tenga las siguientes primitivas:

- `int shmget(int id, int size, void **va)`

Descripción: Obtiene/crea un bloque de memoria compartido de tamaño dado. En el caso que la key no exista, el bloque deberá ser creado (allocated) y mapeado. En otro caso, deberá mapearlo.

Argumentos: `id` es el identificador del recurso, `size` es el tamaño del bloque.

Retorno: El descriptor del shm o -1 en caso de error.

Nota: en la primera llamada al sistema `shmget(id, size)` con un nuevo `id` hará que el sistema cree el bloque de memoria compartida. El próximo proceso que haga la misma llamada (posiblemente con diferente `size`) recibirá el descriptor correspondiente del recurso creado previamente por el sistema.

- `int shmat(int shm)`

Descripción: Obtiene la dirección virtual de la base del bloque.

Parámetros: `shm` es el identificador (descriptor) retornado por `shmget()`.

Retorno: Dirección lógica (como un `int` que debería convertirse a `T*`). En caso de falla deberá retornar -1.

- `int shmclose(int shm)`

Descripción: Libera el bloque de memoria obtenido previamente.

Parámetros: `shm` es el identificador (descriptor) retornado por `shmget()`.

Retorno: -1 en caso de error (`key` inválida). Cero en otro caso.

Detalles de implementación

a) El sistema deberá permitir hasta NSHM bloques de memoria compartida y cada proceso hasta NSHM_PROC shared memories (`NSHM_PROC < NSHM`).

b) El descriptor de una tarea debe tener un arreglo de descriptores de shm en uso.

e) Definir las llamadas al sistema en espacio de usuario correspondientes para cada operación.

f) Escribir un par de programas de usuario que usen un shm.

Proyecto 3: Semáforos

Diseñar e implementar llamadas al sistema sobre semáforos n-arios.

- `int semcreate(int id, int init_value)`
- `int semget(int id)`
- `int sem_wait(int id)`
- `int sem_signal(int id)`

- `int sem_close(int id)`

El sistema deberá manejar hasta `NSEM` semáforos y cada proceso hasta `NSEM_PROC` semáforos (`NSEM_PROC < NSEM`).

Implementar estas funciones en `sem.c`.

Implementar dos procesos de usuario que usen semáforos para acceder en forma sincronizada a un bloque de memoria compartida. Hacer el kernel asigne y mapee una página compartida entre ellos (hardcoded en `kmain.c`).

Proyecto 4: Lectura de consola

Implementar la llamada al sistema `int console_gets(char *str)` que bloquee el proceso invocante hasta que se lea una línea de entrada terminada con ENTER.

Actualmente `ed` contiene el syscall `int console_getc(void)` que retorna un caracter pero lo realiza haciendo espera ocupada (pooling). Modificar esta llamada al sistema y el manejo de interrupciones para que opere con interrupciones.

Hacer un programa de usuario de prueba que permita probar las llamadas al sistema.

Proyecto 5: Mensajes entre procesos

Implementar las llamadas al sistema `int sendmsg(int pid, char* msg)` y `int recvmsg(int pid, char*msg)` para envío y recepción de mensajes entre procesos.

La comunicación debe ser sincrónica, es decir que si un proceso envía un mensaje y no hay un receptor el otro deberá esperar. La operación de recepción también debe bloquearse hasta que otro proceso envíe un mensaje al receptor.

El argumento `pid` es el *process id* del destino u origen, respectivamente. Un valor de 0 en `pid` significa ANY, es decir que el destino u origen puede ser cualquier proceso.

Hacer programas de ejemplo para probar su funcionamiento.

Proyecto 6: Sistema de archivos

Crear llamadas al sistema para trabajar con archivos. Específicamente, implementar las siguientes llamadas al sistema:

- `int open(char *file_name, int mode)`
- `int read(int fd, char *buffer, int count)`
- `int write(int fd, char *buffer, int count)`
- `int close(int fd)`

El sistema deberá representar el conjunto de archivos abiertos por todos los procesos. Cada proceso deberá mantener el conjunto de descriptores de archivos abiertos. Cuando un proceso termina el kernel deberá cerrar los archivos que aún permanezcan abiertos. Tener en cuenta que un mismo archivo puede estar abierto por varios procesos.

Implementación:

- Implementar las llamadas al sistema sobre archivos en `efs.c`. La escritura en archivos se deberá limitar a escribir en áreas existentes ya el sistema de archivos (embedded fs) no permite extender sus contenidos.
- Representar todo en estructuras de datos estáticas (arreglos) con tamaños constantes (definirlas en `efs.h`).
- Cada archivo abierto mantiene en su estado el *offset* o cursor con la posición actual (0 luego de `open`) en donde se hará la próxima lectura o escritura. Las operaciones `read` y `write` deben actualizar el *offset*.

Proyecto 7: Asignación dinámica de memoria de stack.

EDOS asigna una página de stack en modo usuario a los procesos. Modificar el sistema para que asigne automáticamente una página de stack si el proceso intenta acceder a una dirección de memoria entre el tope corriente y N bytes por encima (direcciones lógicas menores).

El sistema deberá asignar páginas de stack hasta una dirección (`brk`) definida. Luego de eso, cualquier intento de acceso a una dirección entre `brk` y el tope del stack deberá ser considerada *stack overflow*.

Hacer al menos un programa de prueba mostrando la asignación dinámica de stack a medida que se producen faltas de página en el área correspondiente.